Binary Search Trees (BST)

A Binary Search Tree is a binary tree where every node satisfies the BST Property:

For every node n: all keys in the left subtree are < n, and all keys in the right subtree are > n.

In short: left < root < right.

This ordering enables fast lookup, insert, and delete - all in O(h) time, where h = tree height.

Here is a BST with 7 nodes:

{"type": "tree", "nodes": "8:3:10 3:1:6 10:-:14 1:-:- 6:4:- 14:-:- 4:-:-"}

Verify the property: every left descendant is smaller, every right descendant is larger.

Search Operation

To search for a key k, start at the root and walk down:

- If k == node → - found!
- If k < node → go left
- If k > node → go right
- If we hit null → k is not in the tree

Example: search for 6 in the tree above.

Step 1: Compare 6 with root 8 → . 6 < 8 → go left to 3.

Step 2: compare 6 with 3 → - 6 > 3 → go right to 6.

Step 3: Compare 6 with 6 → equal → found!

We visited only 3 nodes instead of scanning all 7.

Insert Operation

Insertion follows the same path as search. When we reach a null pointer, we place the new node there.

Before inserting 5:

{"type": "tree", "nodes": "8:3:10 3:1:6 10:-:- 1:-:- 6:-:-"}

Insert 5: 5 < 8 → go left to 3. 5 > 3 → go right to 6. 5 < 6 → . go left → - null → place 5 here.

After inserting 5:

{"type": "tree", "nodes": "8:3:10 3:1:6 10:-:- 1:-:- 6:5:- 5:-:-"}

Deletion: Three cases

Case 1 - Leaf Node: Simply remove it. NO restructuring needed.

Example: delete 1 → just delete the node. Done.

Case 2 – One Child: Replace the node with its single child.

Example: delete 6 (which has left child 5) → 5 takes 6's place.

Case 3 – Two Children: Find the in-order successor (smallest node in the right subtree), copy its value up, then delete the successor (which falls into case 1 or 2).

Example: delete 3 → in-order successor is 4 → replace 3 with 4 → delete old 4 (leaf, case 1).

The in-order successor is guaranteed to have at most one child, so the recursion always terminates cleanly.

Time Complexity

All three operations – search, insert, delete – run in $O(h)$, where h is the height of the tree.

In the best case the tree is bushy and $h = \log_2 n$, giving $O(\log n)$.

In the worst case (inserting sorted data) the tree degenerates into a linked list with $h = n$, giving $O(n)$.

This is why self-balancing variants like AVL and Red-Black trees were invented – they guarantee $h = O(\log n)$ after every operation.

Balanced BST → O(log n), Degenerate → O(n)

Key Takeaways:

- BST property: left subtree < node < right subtree
- In-order traversal of a BST yields sorted output
- Search, insert, delete all follow a single root-to-leaf path
- Deletion with two children uses the in-order successor trick
- Always prefer a self-balancing BST in production code